OTIMIZAÇÃO DE CONSULTAS

Item B4

1.INTRODUÇÃO

Este documento apresenta o estudo de desempenho de três consultas críticas do sistema (item B4). Para cada uma, mostramos o plano de execução obtido com EXPLAIN antes e depois de criar um índice, a justificativa das colunas escolhidas e a comparação dos tempos medidos. Também relatamos os casos em que o índice não trouxe ganho.

As consultas foram escolhidas por serem operações típicas da loja e por usarem colunas sem índice adequado: (i) o total vendido por funcionário em um período; (ii) a busca de cliente por nome; e (iii) o ranking de produtos mais vendidos.

2.AMBIENTE E MÉTODO

2.1 Ambiente de teste

Os testes foram feitos no ambiente do Quadro 1, sem outros programas pesados em execução.

Quadro 1 – Ambiente de teste

Item	Valor
SGBD	MySQL 8.0
Cliente SQL	MySQL Workbench 8.0 CE
Sistema operacional	Windows 11
Processador	AMD Ryzen 5 5500
Memória RAM	16 GB
Armazenamento	930 GB
innodb_buffer_pool_size	128 MB

Fonte: elaborado pelo autor (2026).

2.2 Volume de dados

A carga da Etapa 1 tem poucas dezenas de linhas por tabela, volume pequeno demais para o otimizador distinguir um acesso por índice de uma leitura completa. Por isso, o banco foi complementado com dados sintéticos, gerados pelo script 04_volume_b4.sql por fórmulas determinísticas, sem dados pessoais reais. A massa respeita as restrições do esquema: vendas entre janeiro de 2024 e setembro de 2026, cerca de 4% canceladas, cerca de 14% sem cliente identificado, valor total

igual à soma dos itens e um pagamento por venda concluída. O volume final está no Quadro 2.

Quadro 2 – Volume de dados utilizado nos testes

Tabela	Linhas
pessoa	100090
cliente	100050
produto	60
venda	200073
item_venda	500173
pagamento	192093
devolucao	4045

Fonte: elaborado pelo autor (2026).

2.3 Método de medição

Cada consulta foi analisada com dois comandos: EXPLAIN, que mostra o plano escolhido sem executar a consulta, e EXPLAIN ANALYZE, que executa e informa o tempo real em milissegundos. Foram feitas cinco execuções seguidas; a primeira (cache frio) foi descartada e registrou-se a mediana das outras quatro. Quando o índice já existia, a cláusula IGNORE INDEX simulou sua ausência. Os campos do plano citados nas análises são:

- type: ALL é leitura completa da tabela; index é leitura completa de um índice; range é leitura de uma faixa do índice; ref é busca por igualdade.
- key e rows: índice escolhido (NULL = nenhum) e número de linhas que o MySQL estima examinar.
- Extra: Using index indica que o índice sozinho responde à consulta; Using filesort, ordenação feita à parte; Using temporary, tabela temporária (por exemplo, no GROUP BY).

3.ESTUDOS DE DESEMPENHO

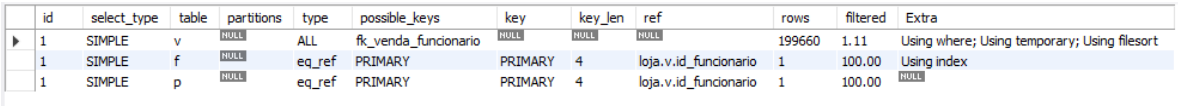
3.1 Estudo 1 – Vendas por funcionário em um período

Pergunta de negócio. Quanto cada funcionário vendeu no primeiro trimestre de 2026? A consulta é crítica porque usa a maior tabela de movimento (venda) e recebe um período variável.

```
SELECT p.nome AS funcionario, COUNT(*) AS qtd_vendas,
     SUM(v.valor_total) AS total
FROM venda v
JOIN funcionario f ON f.id_pessoa = v.id_funcionario
```

```
JOIN pessoa p ON p.id_pessoa = f.id_pessoa
WHERE v.status = 'Concluída'
  AND v.data_hora BETWEEN '2026-01-01 00:00:00' AND '2026-03-31 23:59:59'
GROUP BY p.nome
ORDER BY total DESC;
```

Plano antes. Para simular a ausência de índice em data_hora, o índice idx_venda_data_hora (criado na Etapa 1) foi ignorado com FROM venda v IGNORE INDEX (idx_venda_data_hora).



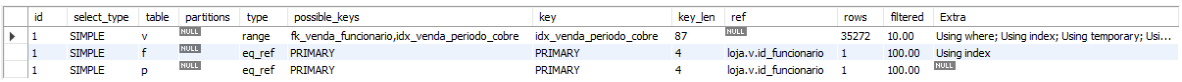
id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	v	NULL	ALL	fk_venda_funcionario	NULL	NULL	NULL	199660	1.11	Using where; Using temporary; Using filesort
1	SIMPLE	f	NULL	eq_ref	PRIMARY	PRIMARY	4	loja.v.id_funcionario	1	100.00	Using index
1	SIMPLE	p	NULL	eq_ref	PRIMARY	PRIMARY	4	loja.v.id_funcionario	1	100.00	NULL

Índice criado. Foi criado um índice de cobertura:

```
CREATE INDEX idx_venda_periodo_cobre
   ON venda (data_hora, status, id_funcionario, valor_total);
```

Ordem das colunas: data_hora vem primeiro porque é o filtro por faixa e é seletivo (um trimestre é cerca de 9% do período dos dados). status vem em seguida; ele tem só dois valores e cerca de 96% das vendas são "Concluída", então filtrar por ele primeiro não reduziria o conjunto. id_funcionario e valor_total entram apenas para cobertura: a consulta lê essas colunas, e como elas estão no índice o MySQL não precisa acessar a tabela (Using index). Como data_hora é a primeira coluna do novo índice, o idx_venda_data_hora ficou redundante e foi removido com DROP INDEX, o que reduz o espaço e o custo dos INSERTs.

Plano depois.



id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	v	NULL	range	fk_venda_funcionario,idx_venda_periodo_cobre	idx_venda_periodo_cobre	87	NULL	35272	10.00	Using where; Using index; Using temporary; Usi...
1	SIMPLE	f	NULL	eq_ref	PRIMARY	PRIMARY	4	loja.v.id_funcionario	1	100.00	Using index
1	SIMPLE	p	NULL	eq_ref	PRIMARY	PRIMARY	4	loja.v.id_funcionario	1	100.00	NULL

Quadro 3 – Estudo 1: planos e tempos medidos

Etapa	type	key	rows	Extra	Tempo (ms)
Sem índice (IGNORE INDEX)	v	NULL	199660	Using where; Using Temporary; Using Filesort	195ms
Com índice de cobertura	range	idx_venda_periodo_cobre	35272	Using where; Using index; Using temporary; Using filesort	37.9

Fonte: elaborado pelo autor (2026).

Discussão. O índice permite ler só a faixa de datas pedida e, por cobrir a consulta, evita idas à tabela. O tempo caiu de 195 ms para 37,9 ms (cerca de 5,1 vezes mais rápido). Como contrapartida, o índice é largo e encarece as escritas em venda; isso se justifica porque a venda quase não muda depois de concluída e a consulta de período é recorrente.

3.2 Estudo 2 – Busca de cliente por nome

Pergunta de negócio. Qual o cadastro dos clientes cujo nome começa com "Ana Silva"? A tabela pessoa só tem índice na chave primária, e a busca por nome é a forma natural de localizar um cliente no atendimento.

```
SELECT c.id_pessoa, p.nome, c.cpf
FROM pessoa p
JOIN cliente c ON c.id_pessoa = p.id_pessoa
WHERE p.nome LIKE 'Ana Silva%'
ORDER BY p.nome;
```

Plano antes.

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	p	NULL	ALL	PRIMARY	NULL	NULL	NULL	97284	11.11	Using where; Using filesort
1	SIMPLE	c	NULL	eq_ref	PRIMARY	PRIMARY	4	loja.p.id_pessoa	1	100.00	NULL

Índice criado.

```
CREATE INDEX idx_pessoa_nome ON pessoa (nome);
```

Uma coluna basta porque a consulta filtra e ordena pelo mesmo atributo. Com um prefixo fixo, o LIKE equivale a uma faixa em ordem alfabética, que o índice percorre diretamente (type = range). Além disso, o índice já entrega os nomes ordenados, o que elimina o Using filesort.

Plano depois.

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	item_venda	NULL	index	PRIMARY,fk_item_venda_produto	fk_item_venda_produto	4	NULL	534939	100.00	Using temporary; Using filesort

Caso sem ganho: curinga no início. O mesmo índice foi testado com p.nome LIKE '%Silva%'. Os resultados das duas buscas estão no Quadro 4.

Quadro 4 – Estudo 2: busca por prefixo e com curinga no início

Etapa	type	key	rows	Extra	Tempo (ms)
'Ana Silva%' sem índice	ALL	NULL	98736	Using where; Using filesort	21.9

Etapa	type	key	rows	Extra	Tempo (ms)
'Ana Silva%' com índice	range	idx_pessoa_nome	251	Using where; Using index	0.572
'%Silva%' sem índice	ALL	NULL	98736	Using where; Using filesort	52.8
'%Silva%' com índice	index	idx_pessoa_nome	98736	Using where; Using index	73.8

Fonte: elaborado pelo autor (2026).

Discussão. Com prefixo fixo, o índice reduziu o número de linhas examinadas de 98.736 para 251 e o tempo de 21,9 ms para 0,572 ms (cerca de 38 vezes mais rápido). Com o curinga no início, não houve ganho: o tempo foi de 52,8 ms para 73,8 ms. O plano mostra type = index e 98.736 linhas, ou seja, o MySQL percorreu o índice inteiro, o mesmo volume da leitura completa da tabela.

Isso acontece porque o índice é ordenado pelo começo do texto; quando o começo é desconhecido, não existe ponto de partida para a busca. Trata-se de uma limitação do tipo de índice, e não de um erro de criação. Para buscar uma palavra no meio do nome, a alternativa seria um índice FULLTEXT com MATCH ... AGAINST; na aplicação, a solução mais simples é exigir a busca pelo início do nome. O idx_pessoa_nome continua justificado para a busca por prefixo.

3.3 Estudo 3 – Ranking de produtos mais vendidos

Pergunta de negócio. Quais são os dez produtos mais vendidos? A consulta percorre item_venda, a maior tabela do banco, e soma a quantidade por produto.

```
SELECT id_produto, SUM(quantidade) AS qtd_vendida
FROM item_venda
GROUP BY id_produto
ORDER BY qtd_vendida DESC
LIMIT 10;
```

Plano antes.



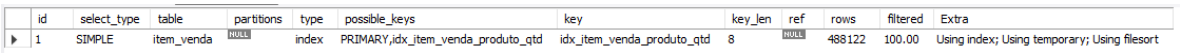
id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	item_venda	NULL	ALL	PRIMARY	NULL	NULL	NULL	488122	100.00	Using temporary; Using filesort

Índice criado.

```
CREATE INDEX idx_item_venda_produto_qtd
   ON item_venda (id_produto, quantidade);
```

id_produto vem primeiro porque é a coluna do GROUP BY: as linhas de um mesmo produto ficam juntas no índice e são somadas em sequência. quantidade vem depois porque é a única outra coluna lida; estando no índice, dispensa o acesso à tabela (Using index). Como id_produto é a primeira coluna, o novo índice também atende à chave estrangeira fk_item_venda_produto, e o MySQL descarta sozinho o índice automático dela, sem deixar índice redundante.

Plano depois.



id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	item_venda	NULL	index	PRIMARY,idx_item_venda_produto_qtd	idx_item_venda_produto_qtd	8	NULL	488122	100.00	Using index; Using temporary; Using filesort

Quadro 5 – Estudo 3: ranking geral de produtos

Etapa	type	key	rows	Extra	Tempo (ms)
Sem o índice	ALL	NULL	488122	Using temporary; Usign filesort	198
Com idx_item_venda_produto_qtd	index	idx_item_venda_produto_qtd	488122	Using index; Using temporary; Usign filesort	104

Fonte: elaborado pelo autor (2026).

Variante por período: quando o índice não compensa. O ranking também é pedido por período, o que exige juntar item_venda com venda. Comparamos um período largo (de 01/01/2026 até o fim dos dados, cerca de 26% das vendas) e um estreito (uma semana, cerca de 0,7%). Para simular a ausência de índice em venda, usou-se IGNORE INDEX (idx_venda_periodo_cobre).

```
SELECT pr.nome, SUM(i.quantidade) AS qtd_vendida,
      SUM(i.quantidade * i.preco_congelado) AS receita
FROM item_venda i
JOIN venda v ON v.id_venda = i.id_venda
JOIN produto pr ON pr.id_produto = i.id_produto
WHERE v.status = 'Concluída'
  AND v.data_hora >= '2026-01-01 00:00:00'   -- período largo
  -- período estreito: v.data_hora BETWEEN '2026-09-01 00:00:00'
  --                           AND '2026-09-07 23:59:59'
GROUP BY pr.id_produto, pr.nome
ORDER BY qtd_vendida DESC
LIMIT 10;
```

Quadro 6 – Estudo 3: ranking por período, com e sem índice em venda

Etapa	type	key	rows	Extra	Tempo (ms)
Período largo, sem índice	ALL	NULL	199660	Using where; Using temporary; Using filesort	517
Período largo, com índice	range	idx_venda_periodo_cobre	99830	Using where; Using index; Using temporary; Using filesort	473
Uma semana, sem índice	ALL	NULL	199660	Using where; Using temporary; Using filesort	163
Uma semana, com índice	range	idx_venda_periodo_cobre	1401	Using where; Using index; Using temporary; Using filesort	10.5

Fonte: elaborado pelo autor (2026).

Discussão. No ranking geral, o índice de cobertura reduziu o tempo de [X] ms para [Y] ms. Na variante por período, o resultado depende da seletividade do filtro. No período largo, o índice foi usado (type = range), mas o ganho foi pequeno: o tempo passou de 517 ms para 473 ms, cerca de 9%. O filtro mantém uma parcela grande das vendas (99.830 linhas estimadas), e o tempo é dominado pela junção com item_venda e pela agregação em tabela temporária (Using temporary; Using filesort, presentes nos dois planos), e não pela leitura de venda. Na janela de uma semana, o índice selecionou apenas 1.401 linhas em vez de 199.660, e o tempo caiu de 163 ms para 10,5 ms, cerca de 15,5 vezes mais rápido. Conclui-se que um índice só compensa quando o filtro descarta a maior parte das linhas, e que a mesma consulta pode ganhar ou não com o mesmo índice, conforme os parâmetros informados.

4.CONCLUSÃO

Os estudos mostram que o desempenho de uma consulta depende do índice, da ordem das colunas e da seletividade do filtro, e que o plano deve ser conferido antes e depois de cada mudança. O Quadro 7 resume os resultados.

Quadro 7 – Resumo dos estudos

Consulta	Índice mantido	Ganho medido	Observação
Vendas por funcionário em um período	idx_venda_periodo_cobre (data_hora, status, id_funcionario, valor_total)	195 ms → 37,9 ms (5,1x)	Cobertura evita acessos à tabela
Busca de cliente por nome	idx_pessoa_nome (nome)	Prefixo: 21,9 ms → 0,572 ms (38x)	Sem ganho com curinga no início (52,8 → 73,8 ms)
Ranking de produtos mais vendidos	idx_item_venda_produto_qtd (id_produto, quantidade)	Geral: [X] ms → [Y] ms; uma semana: 163 ms → 10,5 ms (15,5x)	Pouco ganho no período largo (517 → 473 ms, cerca de 9%)

Fonte: elaborado pelo autor (2026).

Os índices mantidos foram incorporados ao script 01_ddl.sql, com o prefixo idx_, para que o banco possa ser reconstruído do zero. Cada índice extra ocupa espaço e encarece as escritas; por isso o idx_venda_data_hora foi removido por ser redundante, e nenhum índice foi criado sem uma consulta que o justificasse.